Application of RREF using Python

June 12, 2026

Applications of RREF using Python

Ajit Kumar

Institute of Chemical Technology (ICT), Mumbai

1 Interpreting the Matrix Equation $Ax = b$

Suppose

$$A = \begin{bmatrix} | & | & & | \\ a_1 & a_2 & \cdots & a_n \\ | & | & & | \end{bmatrix}$$

has columns $a_1, a_2, \ldots, a_n$, and let

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

Consider the matrix equation

$$Ax = b.$$

We shall see that this equation can be interpreted as a statement about linear combinations of the columns of A.

1.1 Writing Ax as a Linear Combination of the Columns of A

Since

$$A = \begin{bmatrix} | & | & & | \\ a_1 & a_2 & \cdots & a_n \\ | & | & & | \end{bmatrix},$$

we have

$$Ax = \begin{bmatrix} | & | & & | \\ a_1 & a_2 & \cdots & a_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

By the definition of matrix multiplication,

$$Ax = x_1a_1 + x_2a_2 + \cdots + x_na_n.$$

Therefore the equation

$$Ax = b$$

can be rewritten as

$$x_1a_1 + x_2a_2 + \cdots + x_na_n = b.$$

1.2 What Does This Mean?

The vector x contains the coefficients of a linear combination of the columns of A.

Thus solving

$$Ax = b$$

is equivalent to finding scalars

$$x_1, x_2, \ldots, x_n$$

such that

$$b = x_1a_1 + x_2a_2 + \cdots + x_na_n.$$

In other words:

Solving $Ax = b$ means expressing b as a linear combination of the columns of A.

$$Ax = b \text{ has solution iff } b \in \text{span}(\{b_1, \ldots, b_m\}.$$

1.3 Reduced Row Echelon Form (RREF)

The **Reduced Row Echelon Form (RREF)** of a matrix is a simplified form obtained by applying elementary row operations.

A matrix is in RREF if:

1. All zero rows, if any, occur at the bottom.
2. The first nonzero entry in each nonzero row is (1) (called a pivot).
3. Each pivot lies to the right of the pivot in the row above it.
4. Each pivot is the only nonzero entry in its column.

[3]:
```
from sympy import Matrix,symbols
from IPython.display import display
```

[6]:
```
A = Matrix([
    [1, 2, 3, 4],
    [2, 4, 7, 10],
    [3, 6, 10, 14],
    [1, 1, 2, 3]
])
x1,x2,x3,x4 = symbols('x1 x2 x3 x4')
x = Matrix([x1,x2,x3,x4])
```

[7]:
```
A*x
```

[7]:
$$\begin{bmatrix} x_1 + 2x_2 + 3x_3 + 4x_4 \\ 2x_1 + 4x_2 + 7x_3 + 10x_4 \\ 3x_1 + 6x_2 + 10x_3 + 14x_4 \\ x_1 + x_2 + 2x_3 + 3x_4 \end{bmatrix}$$

[22]:
```
cols = [A[:,i] for i in range(4)]

display(cols)
```

```
[Matrix([
 [1],
 [2],
 [3],
 [1]]),
 Matrix([
 [2],
 [4],
 [6],
 [1]]),
 Matrix([
 [ 3],
 [ 7],
 [10],
 [ 2]]),
```

```
Matrix([
[ 4],
[10],
[14],
[ 3]])]
```

[18]:
```
LC = sum([x[i]*A[:,i] for i in range(4)], Matrix.zeros(4,1))

display(LC)
```

$$\begin{bmatrix} x_1 + 2x_2 + 3x_3 + 4x_4 \\ 2x_1 + 4x_2 + 7x_3 + 10x_4 \\ 3x_1 + 6x_2 + 10x_3 + 14x_4 \\ x_1 + x_2 + 2x_3 + 3x_4 \end{bmatrix}$$

1.4 Finding RREF of a matrix

[23]:
```
from sympy import Matrix
from IPython.display import display

A = Matrix([
    [1, 2, 3, 4],
    [2, 4, 7, 10],
    [3, 6, 10, 14],
    [1, 1, 2, 3]
])
display(A)
R, pivots = A.rref()

print("RREF:")
display(R)

print("Pivot columns:")
print(pivots)

print("Rank:")
print(len(pivots))
```

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 7 & 10 \\ 3 & 6 & 10 & 14 \\ 1 & 1 & 2 & 3 \end{bmatrix}$$

```
RREF:
```

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

```
Pivot columns:
(0, 1, 2)
Rank:
3
```

[27]:
```
## Making matrix using vectors and then fining RREF

v1 = Matrix([1, 2, 1, 3])
v2 = Matrix([2, 4, 1, 5])
v3 = Matrix([3,6, 2, 8])
v4 = Matrix([4,10, 3,11])
A = Matrix.hstack(v1, v2, v3, v4)

display(A)
```

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 6 & 10 \\ 1 & 1 & 2 & 3 \\ 3 & 5 & 8 & 11 \end{bmatrix}$$

[28]:
```
R, pivots = A.rref()

print("RREF:")
display(R)

print("Pivot columns:")
print(pivots)

print("Rank:")
print(len(pivots))
```

```
RREF:
```

$$\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

```
Pivot columns:
(0, 1, 3)
Rank:
3
```

[29]:
```
## Creating row matrix from vectors
B = Matrix.vstack(v1.T, v2.T, v3.T,v4.T)
display(B)
```

$$\begin{bmatrix} 1 & 2 & 1 & 3 \\ 2 & 4 & 1 & 5 \\ 3 & 6 & 2 & 8 \\ 4 & 10 & 3 & 11 \end{bmatrix}$$

1.5 Solving a system of linear equations using RREF

[30]:
```
from sympy import Matrix

coeff = Matrix([
    [1,  1,  1,  1],
    [2, -1,  1,  3],
    [1,  2, -1,  1],
    [3,  1,  2, -1]
])

b = Matrix([10,15,6,7])

aug = coeff.row_join(b)
display(aug)
R, pivots = aug.rref()
display(R)
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 10 \\ 2 & -1 & 1 & 3 & 15 \\ 1 & 2 & -1 & 1 & 6 \\ 3 & 1 & 2 & -1 & 7 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 2 \\ 0 & 0 & 1 & 0 & 3 \\ 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

[31]:
```
s = coeff.solve(b)
display(s)
```

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

1.6 Finidng inverse using RREF

[32]:
```
from sympy import Matrix, eye

A = Matrix([
    [1,  1,  1,  1],
    [2, -1,  1,  3],
    [1,  2, -1,  1],
```

```
    [3,  1,  2, -1]
])

I = eye(4)
display(I)
Aug = A.row_join(I)

display(Aug)
```

$$\begin{bmatrix}1 & 0 & 0 & 0\\0 & 1 & 0 & 0\\0 & 0 & 1 & 0\\0 & 0 & 0 & 1\end{bmatrix}$$

$$\begin{bmatrix}1 & 1 & 1 & 1 & 1 & 0 & 0 & 0\\2 & -1 & 1 & 3 & 0 & 1 & 0 & 0\\1 & 2 & -1 & 1 & 0 & 0 & 1 & 0\\3 & 1 & 2 & -1 & 0 & 0 & 0 & 1\end{bmatrix}$$

[33]:
```
R, pivots = Aug.rref()

display(R)
```

$$\begin{bmatrix}1 & 0 & 0 & 0 & -\frac{19}{33} & \frac{7}{33} & \frac{8}{33} & \frac{10}{33}\\0 & 1 & 0 & 0 & \frac{17}{33} & -\frac{8}{33} & \frac{5}{33} & -\frac{2}{33}\\0 & 0 & 1 & 0 & \frac{25}{33} & -\frac{4}{33} & -\frac{14}{33} & -\frac{1}{33}\\0 & 0 & 0 & 1 & \frac{10}{33} & \frac{5}{33} & \frac{1}{33} & -\frac{7}{33}\end{bmatrix}$$

[34]:
```
A_inv = R[:, 4:]       # columns 4 through 7

display(A_inv)
```

$$\begin{bmatrix}-\frac{19}{33} & \frac{7}{33} & \frac{8}{33} & \frac{10}{33}\\\frac{17}{33} & -\frac{8}{33} & \frac{5}{33} & -\frac{2}{33}\\\frac{25}{33} & -\frac{4}{33} & -\frac{14}{33} & -\frac{1}{33}\\\frac{10}{33} & \frac{5}{33} & \frac{1}{33} & -\frac{7}{33}\end{bmatrix}$$

[35]:
```
display(A * A_inv)
```

$$\begin{bmatrix}1 & 0 & 0 & 0\\0 & 1 & 0 & 0\\0 & 0 & 1 & 0\\0 & 0 & 0 & 1\end{bmatrix}$$

1.7 Linearly dependent/independent

[38]:
```
from sympy import Matrix

v1 = Matrix([1,1,0,-1,0,1])
v2 = Matrix([2,1,-1,0,1,0])
v3 = Matrix([3,2,-1,-1,1,1])
v4 = Matrix([3,2,-1,1,1,0])
v5 = Matrix([1,1,1,1,3,2])
v6 = Matrix([2,1,-2,0,-2,-2])

M = Matrix.hstack(v1,v2,v3,v4,v5,v6)

display(M)
```

$$\begin{bmatrix} 1 & 2 & 3 & 3 & 1 & 2 \\ 1 & 1 & 2 & 2 & 1 & 1 \\ 0 & -1 & -1 & -1 & 1 & -2 \\ -1 & 0 & -1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 3 & -2 \\ 1 & 0 & 1 & 0 & 2 & -2 \end{bmatrix}$$

[39]:
```
R, p = M.rref()
R
```

[39]:
$$\begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

[40]:
```
v1+v2==v3,v4-v5==v6
```

[40]:
```
(True, True)
```

1.8 Coordinates of a vector using RREF

[106]:
```
from sympy import Matrix

b1 = Matrix([1,1,0,0])
b2 = Matrix([1,0,1,0])
b3 = Matrix([1,0,0,1])
b4 = Matrix([0,1,1,1])

v = Matrix([5,3,4,2])

P = Matrix.hstack(b1,b2,b3,b4)
```

```
Aug = P.row_join(v)

display(Aug)

R, pivots = Aug.rref()
display(R)
```

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 5 \\ 1 & 0 & 0 & 1 & 3 \\ 0 & 1 & 0 & 1 & 4 \\ 0 & 0 & 1 & 1 & 2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & \frac{5}{3} \\ 0 & 1 & 0 & 0 & \frac{8}{3} \\ 0 & 0 & 1 & 0 & \frac{2}{3} \\ 0 & 0 & 0 & 1 & \frac{4}{3} \end{bmatrix}$$

[107]:
```
coords = P.inv()*v
display(coords)
```

$$\begin{bmatrix} \frac{5}{3} \\ \frac{8}{3} \\ \frac{2}{3} \\ \frac{4}{3} \end{bmatrix}$$

[108]:
```
coords = P.solve(v)
display(coords)
```

$$\begin{bmatrix} \frac{5}{3} \\ \frac{8}{3} \\ \frac{2}{3} \\ \frac{4}{3} \end{bmatrix}$$

1.9 Linear Spans

[41]:
```
v1 = Matrix([1,1,0,-1,0])
v2 = Matrix([2,1,-1,0,1])
v3 = Matrix([3,2,-1,-1,1])
v4 = Matrix([3,2,-1,1,1])
v5 = Matrix([1,1,0,-3,0])
v6 = Matrix([4,3,-1,-4,1])
M = Matrix.hstack(v1,v2,v3,v4,v5,v6).T
M
```

[41]:
$$\begin{bmatrix} 1 & 1 & 0 & -1 & 0 \\ 2 & 1 & -1 & 0 & 1 \\ 3 & 2 & -1 & -1 & 1 \\ 3 & 2 & -1 & 1 & 1 \\ 1 & 1 & 0 & -3 & 0 \\ 4 & 3 & -1 & -4 & 1 \end{bmatrix}$$

[42]:
```
R, p = M.rref()
R
```

[42]:
$$\begin{bmatrix} 1 & 0 & -1 & 0 & 1 \\ 0 & 1 & 1 & 0 & -1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Linear span of the vectors $\{v_1, \dots, v_6\}$ is subspace W with a basis

$$\{(1, 0, -1, 0, 1), (0, 1, 1, 0, 1), (0, 0, 0, 1, 0)\}$$

1.10 Null space (kernel) of A

[43]:
```
M.rref()[0]
```

[43]:
$$\begin{bmatrix} 1 & 0 & -1 & 0 & 1 \\ 0 & 1 & 1 & 0 & -1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$Mx = 0 \iff Rx = 0$$

. That imples

$$x_1 + x_3 + 2x_5 + 3x_6 = 0 x_2 + x_3 + x_5 + 3x_6 = 0 x_4 - x_5 - x_6 = 0$$

1.11 Coordinate Vectors

Let

$$B = \{b_1, b_2, \dots, b_n\}$$

be a basis of a vector space V.

Suppose a vector $v \in V$ can be written as

$$v = c_1 b_1 + c_2 b_2 + \cdots + c_n b_n.$$

The coordinate vector of V relative to the basis B is

$$[v]_B = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}.$$

We now show how this can be expressed using matrix multiplication.

1.12 The Basis Matrix

Place the basis vectors as columns of a matrix:

$$P_B = \begin{bmatrix} | & | & & | \\ b_1 & b_2 & \cdots & b_n \\ | & | & & | \end{bmatrix}.$$

Then

$$v = c_1 b_1 + c_2 b_2 + \cdots + c_n b_n$$

can be written as

$$P_B \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} = v.$$

Since

$$[v]_B = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix},$$

we obtain

$$\boxed{P_B [v]_B = v.}$$

This equation says that multiplying the coordinate vector by the basis matrix recovers the actual vector.

In particular

$$\boxed{[v]_B = P_B^{-1} v.}$$

1.13 Change of Basis Matrix

Suppose V is a finite-dimensional vector space and

$$B = \{b_1, \ldots, b_n\}$$

and

$$C = \{c_1, \ldots, c_n\}$$

are two bases of V.

Given the coordinates of a vector relative to the basis (B), we would like to find its coordinates relative to the basis (C).

The matrix that performs this conversion is called a **change of basis matrix**.

1.14 Basis Matrices

Form the basis matrices

$$P_B = \begin{bmatrix} | & | & & | \\ b_1 & b_2 & \cdots & b_n \\ | & | & & | \end{bmatrix}, \qquad P_C = \begin{bmatrix} | & | & & | \\ c_1 & c_2 & \cdots & c_n \\ | & | & & | \end{bmatrix}.$$

For every vector $v \in V$,

$$v = P_B[v]_B$$

and

$$v = P_C[v]_C.$$

1.15 Deriving the Change of Basis Formula

Since

$$P_B[v]_B = P_C[v]_C,$$

multiplying by P_C^{-1} gives

$$[v]_C = P_C^{-1} P_B[v]_B.$$

Thus the matrix

$$\boxed{M_{C \leftarrow B} = P_C^{-1} P_B}$$

converts B-coordinates into C-coordinates.

It is called the **change of basis matrix from B to C.**

[112]:
```
# Basis B
b1 = Matrix([1,1,0,0])
b2 = Matrix([1,0,1,0])
b3 = Matrix([1,0,0,1])
b4 = Matrix([0,1,1,1])

PB = Matrix.hstack(b1,b2,b3,b4)
```

```
# Basis C
c1 = Matrix([1,0,0,0])
c2 = Matrix([1,1,0,0])
c3 = Matrix([1,1,1,0])
c4 = Matrix([1,1,1,1])

PC = Matrix.hstack(c1,c2,c3,c4)

M = PC.inv() * PB
print('The matrix of change of basis is: ')
display(M)
```

The matrix of change of basis is:

$$\begin{bmatrix} 0 & 1 & 1 & -1 \\ 1 & -1 & 0 & 0 \\ 0 & 1 & -1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

1.16 Row and Columns Space

[109]:
```
v1 = Matrix([1, 2, 1, 3])
v2 = Matrix([2, 4, 1, 5])
v3 = Matrix([3,6, 2, 8])
v4 = Matrix([4,10, 3,11])

A = Matrix.hstack(v1, v2, v3, v4)

R, pivots = A.rref()

display(R)
print("Pivot columns:", pivots)
```

$$\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Pivot columns: (0, 1, 3)

[110]:
```
R, pivots = A.T.rref()

display(R)
print("Pivot columns:", pivots)
```

$$\begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

```
Pivot columns: (0, 1, 2)
```

1.17 Column Spavce

[82]:
```
col_basis = [A.col(i) for i in pivots]

for v in col_basis:
    display(v)
```

$$\begin{bmatrix} 1 \\ 2 \\ 1 \\ 3 \end{bmatrix}$$

$$\begin{bmatrix} 2 \\ 4 \\ 1 \\ 5 \end{bmatrix}$$

$$\begin{bmatrix} 3 \\ 6 \\ 2 \\ 8 \end{bmatrix}$$

1.18 Row and Columns Spaces

[49]:
```
v1 = Matrix([1,1,0,-1,0,1])
v2 = Matrix([2,1,-1,0,1,0])
v3 = Matrix([3,2,-1,-1,1,1])
v4 = Matrix([3,2,-1,1,1,0])
v5 = Matrix([1,1,1,1,3,2])
v6 = Matrix([2,1,-2,0,-2,-2])

A = Matrix.hstack(v1,v2,v3,v4,v5,v6)

display(A)
```

$$\begin{bmatrix} 1 & 2 & 3 & 3 & 1 & 2 \\ 1 & 1 & 2 & 2 & 1 & 1 \\ 0 & -1 & -1 & -1 & 1 & -2 \\ -1 & 0 & -1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 3 & -2 \\ 1 & 0 & 1 & 0 & 2 & -2 \end{bmatrix}$$

[50]:
```
R, pivots = A.rref()

display(R)
print("Pivot columns:", pivots)
```

$$\begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

```
Pivot columns: (0, 1, 3, 4)
```

[51]:
```
A.rowspace()
```

[51]:
```
[Matrix([[1, 2, 3, 3, 1, 2]]),
 Matrix([[0, -1, -1, -1, 0, -1]]),
 Matrix([[0, 0, 0, -2, -2, 0]]),
 Matrix([[0, 0, 0, 0, -1, 1]])]
```

[52]:
```
for i, r in enumerate(A.rowspace(), start=1):
    print(f"Basis row {i}")
    display(r)
```

```
Basis row 1
```

$$\begin{bmatrix} 1 & 2 & 3 & 3 & 1 & 2 \end{bmatrix}$$

```
Basis row 2
```

$$\begin{bmatrix} 0 & -1 & -1 & -1 & 0 & -1 \end{bmatrix}$$

```
Basis row 3
```

$$\begin{bmatrix} 0 & 0 & 0 & -2 & -2 & 0 \end{bmatrix}$$

```
Basis row 4
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 & -1 & 1 \end{bmatrix}$$

[85]:
```
N_A = A.nullspace()
print(N_A)
```

```
[Matrix([
[-1],
[-1],
[ 1],
[ 0],
[ 0],
[ 0]]), Matrix([
[ 0],
[ 0],
```

```
 [ 0],
 [-1],
 [ 1],
 [ 1]])]
```

[86]:
```
A.columnspace()
```

[86]:
```
[Matrix([
 [ 1],
 [ 1],
 [ 0],
 [-1],
 [ 0],
 [ 1]]),
 Matrix([
 [ 2],
 [ 1],
 [-1],
 [ 0],
 [ 1],
 [ 0]]),
 Matrix([
 [ 3],
 [ 2],
 [-1],
 [ 1],
 [ 1],
 [ 0]]),
 Matrix([
 [1],
 [1],
 [1],
 [1],
 [3],
 [2]])]
```

[47]:
```
for i, v in enumerate(A.columnspace(), start=1):
    print(f"Basis vector {i}")
    display(v)
```

```
Basis vector 1
```

$$\begin{bmatrix} 1 \\ 1 \\ 0 \\ -1 \\ 0 \\ 1 \end{bmatrix}$$

Basis vector 2

$$\begin{bmatrix} 2 \\ 1 \\ -1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Basis vector 3

$$\begin{bmatrix} 3 \\ 2 \\ -1 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

Basis vector 4

$$\begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 3 \\ 2 \end{bmatrix}$$

1.19 Matrix of a linear transformation

$$T(x_1, x_2, x_3, x_4) = (x_1 + x_2,\ x_2 + x_3,\ x_3 + x_4,\ x_1 + x_4).$$

Find the matrix of T w.r.t. a bases

$$\beta = \{(1,1,0,0), (1,0,1,0), (1,0,0,1), (0,1,1,1)\}$$

on domain and

$$\gamma = \{(1,0,0,0, (1,1,0,0), (1,1,1,0), (1,1,1,1)\}$$

of the codomain.

[53]:
```
# Domain basis
b1 = Matrix([1,1,0,0])
b2 = Matrix([1,0,1,0])
b3 = Matrix([1,0,0,1])
b4 = Matrix([0,1,1,1])

B = Matrix.hstack(b1,b2,b3,b4)

# Codomain basis
c1 = Matrix([1,0,0,0])
c2 = Matrix([1,1,0,0])
c3 = Matrix([1,1,1,0])
```

```
c4 = Matrix([1,1,1,1])

C = Matrix.hstack(c1,c2,c3,c4)
```

[54]:
```
def T(v):
    x1,x2,x3,x4 = v
    return Matrix([
        x1+x2,
        x2+x3,
        x3+x4,
        x1+x4
    ])
```

[55]:
```
columns = []

for b in [b1,b2,b3,b4]:

    Tb = T(b)

    Aug = C.row_join(Tb)

    R,_ = Aug.rref()

    coord = R[:,4]

    columns.append(coord)

TCB = Matrix.hstack(*columns)

display(TCB)
```

$$\begin{bmatrix} 1 & 0 & 1 & -1 \\ 1 & 0 & -1 & 0 \\ -1 & 0 & -1 & 1 \\ 1 & 1 & 2 & 1 \end{bmatrix}$$

[56]:
```
TB = Matrix.hstack(
    T(b1),
    T(b2),
    T(b3),
    T(b4)
)

display(TB)
```

$$\begin{bmatrix} 2 & 1 & 1 & 1 \\ 1 & 1 & 0 & 2 \\ 0 & 1 & 1 & 2 \\ 1 & 1 & 2 & 1 \end{bmatrix}$$

[57]:
```
Aug = C.row_join(TB)

display(Aug)
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 2 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 2 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 & 2 \\ 0 & 0 & 0 & 1 & 1 & 1 & 2 & 1 \end{bmatrix}$$

[58]:
```
R, pivots = Aug.rref()

display(R)
```

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 & -1 \\ 0 & 1 & 0 & 0 & 1 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 & -1 & 0 & -1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 2 & 1 \end{bmatrix}$$

[59]:
```
MatT=R[:, 4:]
MatT
```

[59]:
$$\begin{bmatrix} 1 & 0 & 1 & -1 \\ 1 & 0 & -1 & 0 \\ -1 & 0 & -1 & 1 \\ 1 & 1 & 2 & 1 \end{bmatrix}$$

1.20 Finding linear transformation defined on a basis.

Fix a basis $\beta = \{v_1 = (1,1,0), v_2 = (0,1,1), v_3 = (1,0,1)\}$ of $\mathbb{R}^3$ and three vectors $w_1 = (1,1,1)$, $w_2 = (1,2,3)$ and $w_3 = (1,-2,1)$. Find a linear transformation $T\colon \mathbb{R}^3 \to \mathbb{R}^3$ such that $T(v_i) = w_i$ for $1 \leq i \leq 3$. Do this step by step in Python.

[61]:
```
v1 = Matrix([1,1,0])
v2 = Matrix([0,1,1])
v3 = Matrix([1,0,1])

w1 = Matrix([1,1,1])
w2 = Matrix([1,2,3])
w3 = Matrix([1,-2,1])

PB = Matrix.hstack(v1,v2,v3)
PC = Matrix.hstack(w1,w2,w3)

print("PB =")
display(PB)
```

```
print("PC =")
display(PC)
```

PB =

$$\begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

PC =

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & -2 \\ 1 & 3 & 1 \end{bmatrix}$$

If A is the matrix of T in the standard basis, then

$$A[v_1\ v_2\ v_3] = [T(v_1)\ T(v_2)\ T(v_3)] = [w_1\ w_2\ w_3].$$

[62]:
```
from sympy import eye

Aug = PB.row_join(eye(3))

print("Augmented matrix [V | I]")
display(Aug)
```

Augmented matrix [V | I]

$$\begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

[63]:
```
R, pivots = Aug.rref()

display(R)
```

$$\begin{bmatrix} 1 & 0 & 0 & \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \\ 0 & 1 & 0 & -\frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 & \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \end{bmatrix}$$

[64]:
```
PBinv = R[:,3:]

print("V inverse =")
display(PBinv)
```

V inverse =

$$\begin{bmatrix} \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \\ -\frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \end{bmatrix}$$

[66]:
```
A = PC*PBinv

print("Matrix of T in the standard basis:")
display(A)
```

```
Matrix of T in the standard basis:
```

$$\begin{bmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ -\frac{3}{2} & \frac{5}{2} & -\frac{1}{2} \\ -\frac{1}{2} & \frac{3}{2} & \frac{3}{2} \end{bmatrix}$$

[67]:
```
print("A*v1 =")
display(A*v1)

print("A*v2 =")
display(A*v2)

print("A*v3 =")
display(A*v3)
```

```
A*v1 =
```

$$\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

```
A*v2 =
```

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

```
A*v3 =
```

$$\begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

[68]:
```
x1,x2,x3 = symbols('x1 x2 x3')

x = Matrix([x1,x2,x3])

display(A*x)
```

$$\begin{bmatrix} \frac{x_1}{2} + \frac{x_2}{2} + \frac{x_3}{2} \\ -\frac{3x_1}{2} + \frac{5x_2}{2} - \frac{x_3}{2} \\ -\frac{x_1}{2} + \frac{3x_2}{2} + \frac{3x_3}{2} \end{bmatrix}$$

[ ]:
```

```

1.21 Finding the linear transformation whose matrix w.r.t. given bases is given

Find the linear transformation $T : \mathbb{R}^3 \to \mathbb{R}^3$ whose matrix with respecto to bases

$$B = \{b_1 = (1,1,0,0), b_2 = (1,0,1,0,), b_3 = (1,0,0,1), b_4 = (0,1,1,1)\}$$

of the domain and

$$C\{c_1 = (1,0,0,0), c_2 = (1,1,0,0), c_3 = (1,1,1,0), c_4 = (1,1,1,1)\}$$

of the codomain is

$$[T]_B^C = \begin{bmatrix} 2 & 1 & 1 & 1 \\ 1 & 1 & 0 & 2 \\ 0 & 1 & 1 & 2 \\ 1 & 1 & 2 & 1 \end{bmatrix} =: M.$$

1.22 Step 1

Each column of M gives the coordinates of $T(b_i)$ with respect to the basis C.

For example,

$$[T(b_1)]_C = \begin{bmatrix} 2 \\ 1 \\ 0 \\ 1 \end{bmatrix}.$$

Therefore

$$T(b_1) = 2c_1 + c_2 + c_4.$$

Similarly, the remaining columns give $T(b_2), T(b_3)$ and $T(b_4)$.

The columns of

$$PC\,M$$

are the vectors

$$T(b_1), \quad T(b_2), \quad T(b_3), \quad T(b_4)$$

expressed in standard coordinates.

Thus

$$[T(b_1)\ T(b_2)\ T(b_3)\ T(b_4)] = PC\,M.$$

1.23 Step 2

Let A be the matrix of T with respect to the standard basis.

Since

$$Ab_i = T(b_i),$$

we have

$$A[b_1\ b_2\ b_3\ b_4] = [T(b_1)\ T(b_2)\ T(b_3)\ T(b_4)].$$

Therefore

$$A\,PB = PC\,M.$$

Multiplying by PB^{-1},

$$A = PC\,M\,PB^{-1}.$$

1.24 Step 3

To obtain the formula for T, multiply A by

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}.$$

Therefore

$$T(x_1, x_2, x_3, x_4) = (2x_1 + 4x_2 + x_3 + x_4,\ 3x_2 + x_4,\ x_1 - x_2 + 2x_3 + 2x_4,\ x_1 + x_4)\,.$$

This is the unique linear transformation whose matrix relative to the domain basis B and codomain basis C is

$$[T]_{C \leftarrow B} = \begin{bmatrix} 2 & 1 & 1 & 1 \\ 1 & 1 & 0 & 2 \\ 0 & 1 & 1 & 2 \\ 1 & 1 & 2 & 1 \end{bmatrix}.$$

[70]:
```
# Domain basis
b1 = Matrix([1,1,0,0])
b2 = Matrix([1,0,1,0])
b3 = Matrix([1,0,0,1])
b4 = Matrix([0,1,1,1])
```

```
B1 = [b1,b2,b3,b4]
PB = Matrix.hstack(b1,b2,b3,b4)

# Codomain basis
c1 = Matrix([1,0,0,0])
c2 = Matrix([1,1,0,0])
c3 = Matrix([1,1,1,0])
c4 = Matrix([1,1,1,1])
C1 = [c1,c2,c3,c4]
PC = Matrix.hstack(c1,c2,c3,c4)

M = Matrix([[1, 0, 1, -1], [1, 0, -1, 0], [-1, 0, -1, 1], [1, 1, 2, 1]])

## Symbolic vector x = (x1, x2, x3, x4)

x1,x2,x3,x4 = symbols('x1 x2 x3 x4')

x = Matrix([x1,x2,x3,x4])

Tx = PC * M * PB.inv() * x

display(Tx)
```

$$\begin{bmatrix} x_1 + x_2 \\ x_2 + x_3 \\ x_3 + x_4 \\ x_1 + x_4 \end{bmatrix}$$

1.25 Rank-Nullity Theorem

1.26 Example: Rank, Null Space and the Rank–Nullity Theorem

Consider the linear transformation

$$T : \mathbb{R}^5 \to \mathbb{R}^4$$

defined by

$$T(x) = Ax,$$

where

$$A = \begin{bmatrix} 1 & 0 & 1 & 2 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 2 & 3 & 1 \\ 1 & 2 & 3 & 4 & 1 \end{bmatrix}.$$

Perform the following tasks:

1. Compute the reduced row echelon form (RREF) of A.
2. Determine the rank of T.
3. Find a basis for the column space of A.
4. Find a basis for the null space of A.
5. Determine the nullity of T.
6. Verify the Rank–Nullity Theorem

$$\text{rank}(T) + \text{nullity}(T) = \dim(\mathbb{R}^5).$$

7. Verify that each basis vector of the null space indeed belongs to the kernel of T.

[84]:
```
A = Matrix([
    [1,0,1,2,0],
    [0,1,1,1,1],
    [1,1,2,3,1],
    [1,2,3,4,1]
])

display(A)
```

$$\begin{bmatrix} 1 & 0 & 1 & 2 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 2 & 3 & 1 \\ 1 & 2 & 3 & 4 & 1 \end{bmatrix}$$

[85]:
```
print("Rank =", A.rank())
```

```
Rank = 3
```

[86]:
```
R, pivots = A.rref()

display(R)
print("Pivot columns:", pivots)
```

$$\begin{bmatrix} 1 & 0 & 1 & 2 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

```
Pivot columns: (0, 1, 4)
```

[87]:
```
for v in A.columnspace():
    display(v)
```

$$\begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

[88]:
```
for v in A.nullspace():
    display(v)
```

$$\begin{bmatrix} -1 \\ -1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} -2 \\ -1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

[ ]:
```

```